```
> pact@1.0.0 publish:pact
> pact-broker publish ./pacts --consumer-app-version=1.0.2 --tag=main --broker-base-url=$PACT_BROKER_BASE_
URL --broker-token=$PACT_BROKER_TOKEN

Updated GraphQLConsumer version 1.0.2 with tags main
  Next steps:
    Configure the version branch to be the value of your repository branch.
Pact successfully republished for GraphQLConsumer version 1.0.2 and provider GraphQLProvider with no conte
nt changes.
  View the published pact at https://mariecruz.pactflow.io/pacts/provider/GraphQLProvider/consumer/GraphQL
Consumer/version/1.0.2
  Events detected: contract_published
  Next steps:
    * Configure separate GraphQLProvider pact verification build and webhook to trigger it when the pact c
ontent changes. See https://docs.pact.io/go/webhooks
Updated WebConsumer version 1.0.2 with tags main
  Next steps:
    Configure the version branch to be the value of your repository branch.
Pact successfully republished for WebConsumer version 1.0.2 and provider MoviesAPI with no content changes
.
  View the published pact at https://mariecruz.pactflow.io/pacts/provider/MoviesAPI/consumer/WebConsumer/v
ersion/1.0.2
  Events detected: contract_published
  Next steps:
    * Configure separate MoviesAPI pact verification build and webhook to trigger it when the pact content
 changes. See https://docs.pact.io/go/webhooks
```

Figure 7.6 Output of publishing the contracts to PactFlow

7.4 Provider contract tests between a consumer and GraphQL layer

In chapter 6, we introduced how to write the provider contract tests between a web consumer and a REST API. The setup for provider contract testing between the GraphQL consumer and GraphQL provider is similar. One thing to note, though, is that our GraphQL server depends on the movies API, as shown in figure 7.7.